Core Concepts: MCP Servers

> Part of the MCP Server curriculum. This document covers what MCP is and the problem it solves, the host/client/server roles, the three primitives, transports, and a minimal working server.

1. The problem MCP solves

Before a standard protocol existed for it, every LLM application that wanted to call out to external tools or data had to write its own bespoke integration code for every data source it wanted to support — an app supporting five tools against three different LLM hosts meant up to fifteen separate, non-reusable integrations. This is the same **M×N integration problem** that a standard wire protocol always exists to solve: once every server and every host speak the same protocol, any compliant server works with any compliant host, and the two sides can be built, tested, and evolved independently.

MCP (Model Context Protocol) is that standard: a protocol for connecting an LLM application to external tools, data, and prompt templates through one consistent interface. Building a dedicated MCP server around a data source, rather than hand-wiring that data source into one specific LLM app, buys exactly the same thing a dedicated API layer buys a frontend: a stable, reusable contract, usable by more than one consumer, that's independent of how either side is implemented — the same loose-coupling-via-explicit-contracts principle discussed in the architecture overview, applied one layer further out.

2. Roles: host, client, server

Role	What it is
Host	The LLM application itself — a desktop app, an IDE integration, or a custom agent process. This is what a person (or another system) actually interacts with.
Client	The piece living inside the host that speaks the MCP protocol to one specific server — a host maintains one client per server connection.
Server	The process exposing tools, resources, and/or prompts — the thing this curriculum is about building.

A single host can hold connections to several servers at once (a filesystem server, a database server, a domain-specific API-wrapping server like the ones this curriculum builds), and a single server can be reused by several different hosts without any changes on the server side.

3. The three primitives

MCP defines three distinct kinds of thing a server can expose, and it matters which one fits a given use case:

Primitive	Controlled by	What it's for
Tools	The **model** — the LLM decides when to call one, with what arguments	Actions/functions the model can invoke to fetch data or cause an effect, analogous to a function call
Resources	The **application/host**	Read-only data (files, records) the host can choose to attach to context — never invoked as a function call the model reasons about turn-by-turn
Prompts	The **user**	Reusable, user-selected templates a host can surface directly (e.g. as a slash command)

This curriculum focuses on **tools**, since a server that wraps a data-serving API to answer open-ended analytical questions is almost entirely tool-shaped: the model needs to actively decide, per question, which data to fetch and with what arguments — not passively receive a fixed document, and not offer the user a canned template.

4. Protocol basics

MCP messages are **JSON-RPC 2.0** — a lightweight, well-established RPC format: a request carries a method name and parameters and gets a matching response carrying a result or an error; a notification is a one-way message with no response expected.

A connection begins with an **initialize handshake**: the client and server exchange protocol version and capability information (which primitives each side supports) before any real tool-calling traffic flows. You rarely write this handshake by hand — an SDK (§7) handles it — but it's worth knowing it exists, since a version or capability mismatch surfaces here, at connection time, rather than as a mysterious failure on the first real tool call.

5. Transports

The protocol itself is transport-agnostic; two transports cover the vast majority of real use:

Transport	How it works	Fits
stdio	The server runs as a subprocess of the host; messages flow over the subprocess's stdin/stdout	Local-only use — a desktop LLM app or a local CLI tool launching the server itself. No network exposure at all, which is a real security property, not just a simplicity one.
Streamable HTTP	The server runs as a standalone, long-lived process reachable over ordinary HTTP; the current recommended transport for anything remote	Any consumer that isn't co-located with the server — a separately-deployed agent, a shared server used by multiple people. Works behind normal HTTP infrastructure (reverse proxies, TLS termination) the same way any other web service does.

An older SSE (Server-Sent Events)-based transport predates Streamable HTTP and has been superseded by it for new servers — mentioned here only so it's recognizable if encountered in older examples, not as a current recommendation.

Which transport a given deployment needs is a direct consequence of where its clients run — see Deployment, §1 for the concrete decision.

6. A minimal tool-serving server

Most MCP SDKs offer a decorator-based ergonomic layer over the raw protocol — you write plain functions, and the SDK handles JSON-RPC framing, schema generation from type hints, and the initialize handshake:

```python
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("example-server")

@mcp.tool()
def get_widget(widget_id: str) -> dict:
    """Look up a single widget by its id."""
    return {"id": widget_id, "status": "ok"}

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

The function's type hints become the tool's input JSON schema automatically (the same type-hints-drive-validation idea FastAPI uses for request/response models), and the docstring becomes the tool's description — the text a model actually reads to decide whether and how to call it (see Tool Design, §1 for why that description text matters as much as the schema itself).

7. What a tool call looks like on the wire

Demystified, a simplified version of the two messages a `get_widget("abc123")` call actually exchanges:

```
// client -> server
{"jsonrpc": "2.0", "id": 1, "method": "tools/call",
 "params": {"name": "get_widget", "arguments": {"widget_id": "abc123"}}}

// server -> client
{"jsonrpc": "2.0", "id": 1,
 "result": {"content": [{"type": "text", "text": "{\"id\": \"abc123\", \"status\": \"ok\"}"}],
            "isError": false}}
```

Two things worth internalizing from this shape: the result's `content` is a **list of content blocks** (usually text, but a server can return multiple blocks or other block types), not a bare return value — this matters directly when a client-side adapter later has to unwrap it (see Deployment, §3 and the conversational-agent curriculum's testing document for a concrete case where assuming a plain string instead caused a real bug); and `isError` is how a tool signals failure — a distinct, structured channel a model can react to, not a thrown exception that would otherwise crash the connection (see Tool Design, §5).

8. Why a dedicated server, instead of the model just calling the API directly

A model *could*, in principle, be handed raw API documentation and asked to construct HTTP calls itself — but a purpose-built MCP server buys several things that approach doesn't:

- **Tool schemas can be written for a model's reasoning, not a REST client's plumbing.** A parameter name, description, and allowed-value set can be worded exactly for what a model needs to decide correctly, independent of whatever the underlying API's own parameter names happen to be.
- **Composition.** One tool can internally combine several underlying calls to answer one question a user is actually likely to ask, instead of forcing the model to figure out and chain several raw calls itself (see Tool Design, §2).
- **A place to put guardrail logic the raw API doesn't have** — argument resolution, response-size trimming, and structured error messages a model can act on, all covered in the next document.

9. Core libraries summary

Library/tool	Used for
An MCP SDK (e.g. the official Python `mcp` package, or a higher-level wrapper like `FastMCP`)	Protocol framing, schema generation from type hints, transport handling
A fuzzy-matching library (e.g. `rapidfuzz`)	Argument-resolution guards for free-text identifiers — see Tool Design, §3
An HTTP client (e.g. `httpx`)	When the server wraps a REST API rather than a database or filesystem directly

See also

- MCP Server curriculum index
- Tool Design — designing the tools this document showed how to build
- Python API Backend, §1 for the parallel reasoning behind a dedicated API layer
- Conversational Agents, Core Concepts for how an agent actually uses a server built this way